EasyFood — Atividade 7

Implementando o cardápio e o processamento de imagens

Nesta atividade, vamos continuar evoluindo a EasyFood, adicionando o cadastro de pratos aos restaurantes e permitindo o envio de imagens para cada prato.

Na atividade anterior, implementamos um serviço compartilhado de envio de e-mails. Agora vamos trabalhar uma nova necessidade da aplicação: **o gerenciamento do cardápio dos restaurantes**.

Além de cadastrar os pratos no banco de dados, precisaremos receber imagens, reduzir seu tamanho e armazenar a versão otimizada.

Durante a aula, discutimos diferentes possibilidades arquiteturais para o processamento de imagens, incluindo Serverless e FaaS.

Decisão arquitetural: neste momento, manteremos o processamento de imagens dentro do servidor da EasyFood.

Como nosso projeto está em fase acadêmica e possui uma demanda limitada, vamos priorizar a simplicidade, reutilizando a infraestrutura que já temos.

Não será necessário implementar microsserviços, filas ou funções Serverless.

```
Restaurant
    |
    v
Dish
    |
    v
Upload de imagem
    |
    v
Processamento na API EasyFood
    |
    v
Imagem otimizada
```

Parte 1 — PREPARE

1. Deixe sua EasyFood no mesmo ponto da atividade anterior

Antes de começar, confirme que a aplicação está funcionando e que as funcionalidades anteriores continuam operando corretamente.

Sua aplicação já deverá possuir:

- Cadastro e consulta de restaurantes.
- Persistência utilizando Prisma e PostgreSQL.
- Autenticação com JWT.
- Middleware para proteger operações.
- Serviço compartilhado de envio de e-mails.

A estrutura deverá ser semelhante a:

```
src/
■■■ database/
■   ■■■ prisma.js
■■■ modules/
■   ■■■ restaurants/
■   ■   ■■■ restaurant.routes.js
■   ■   ■■■ restaurant.controller.js
■   ■   ■■■ restaurant.service.js
■   ■■■ auth/
```

```
■   ■■■ shared/
■       ■■■ email.service.js
■■■ app.js
■■■ server.js
```

Execute:

```
node server.js
```

Confirme que:

- **GET /restaurants** funciona.
- **POST /restaurants** permite cadastrar restaurantes com autenticação.
- O login retorna um token JWT.
- O envio de e-mail implementado na atividade anterior continua funcionando.

Não remova nem substitua as funcionalidades anteriores. Vamos evoluir a aplicação existente.

Parte 2 — ENTENDA A NOVA NECESSIDADE

2. O Product Owner trouxe uma nova funcionalidade

Atualmente, a EasyFood permite cadastrar restaurantes.

Entretanto, ainda não conseguimos cadastrar os pratos que esses restaurantes oferecem.

Agora precisamos permitir que cada restaurante tenha seu próprio cardápio.

Um prato deverá possuir:

```
name          -> Nome do prato
description   -> Descrição
price         -> Preço
restaurantId  -> Restaurante ao qual pertence
```

Exemplo:

```
{
  "name": "Pizza Margherita",
  "description": "Molho de tomate, queijo e manjericão",
  "price": 49.90
}
```

Antes de implementar, pense:

- Um restaurante pode possuir vários pratos?
- Um prato precisa estar associado a um restaurante?
- Onde devemos colocar essa nova responsabilidade?
- Precisamos criar um novo módulo?
- Podemos reutilizar a arquitetura que já construímos?

Nossa decisão: criaremos um módulo **dishes**, mantendo a organização do monólito modular.

Parte 3 — EVOLUA O BANCO DE DADOS

3. Crie o modelo Dish no Prisma

Abra o arquivo:

```
prisma/schema.prisma
```

Precisamos representar o relacionamento:

```
Restaurant
    |
```

```
    | 1
    |
    | N
    v
   Dish
```

Um restaurante pode possuir vários pratos.

Cada prato pertence a um restaurante.

Adicione o modelo:

```
model Dish {
  id           Int      @id @default(autoincrement())
  name         String
  description  String
  price        Decimal  @db.Decimal(10, 2)
  imageUrl     String?
  restaurantId Int

  restaurant Restaurant @relation(
    fields: [restaurantId],
    references: [id]
  )
}
```

Atenção: o exemplo considera que o identificador de **Restaurant** é um **Int**. Se seu projeto utiliza UUID ou outro tipo, adapte o relacionamento.

No modelo **Restaurant**, adicione o campo de relacionamento:

```
dishes Dish[]
```

Não substitua os campos que já existem no modelo **Restaurant**.

Por que adicionamos imageUrl?

Inicialmente, o prato poderá ser cadastrado sem imagem.

Posteriormente, utilizaremos esse campo para armazenar o caminho da imagem otimizada.

O **?** indica que o campo é opcional.

4. Atualize o banco de dados

Execute:

```
npx prisma migrate dev --name add_dishes
```

Depois:

```
npx prisma generate
```

Confirme que a nova tabela foi criada corretamente.

Parte 4 — CRIE O MÓDULO DISHES

5. Organize os arquivos

Dentro de **src/modules/**, crie:

```
dishes/
```

A estrutura deverá ficar:

```
src/
■■■ modules/
    ■■■ restaurants/
    ■■■ auth/
    ■■■ shared/
    ■   ■■■ email.service.js
```

```
    ■■■ dishes/
        ■■■ dish.routes.js
        ■■■ dish.controller.js
        ■■■ dish.service.js
```

Responsabilidades

dish.routes.js

- Define os endpoints.
- Aplica os middlewares necessários.

dish.controller.js

- Recebe os dados da requisição.
- Chama o Service.
- Retorna a resposta HTTP.

dish.service.js

- Implementa as regras da funcionalidade.
- Consulta e grava os pratos utilizando Prisma.

Não devemos colocar toda a implementação dentro de um único arquivo.

Parte 5 — IMPLEMENTE O CADASTRO DE PRATOS

6. Crie o Dish Service

Abra:

`src/modules/dishes/dish.service.js`

Implemente:

```
const prisma = require("../../database/prisma");

async function createDish(restaurantId, data) {
  const restaurant = await prisma.restaurant.findUnique({
    where: {
      id: Number(restaurantId)
    }
  });

  if (!restaurant) {
    const error = new Error("Restaurante não encontrado");
    error.statusCode = 404;
    throw error;
  }

  const dish = await prisma.dish.create({
    data: {
      name: data.name,
      description: data.description,
      price: data.price,
      restaurantId: Number(restaurantId)
    }
  });

  return dish;
}

async function listDishes(restaurantId) {
  return prisma.dish.findMany({
    where: {
      restaurantId: Number(restaurantId)
    }
  });
}
```

```
module.exports = {
  createDish,
  listDishes
};
```

O que estamos fazendo?

- Verificamos se o restaurante existe.
- Criamos o prato relacionado ao restaurante.
- Permitimos consultar os pratos cadastrados.

Importante: no projeto final, também devemos validar os dados recebidos, especialmente nome, preço e identificadores.

Parte 6 — CRIE O CONTROLLER

7. Implemente o Dish Controller

Abra:

```
src/modules/dishes/dish.controller.js
```

Implemente:

```
const dishService = require("./dish.service");

async function create(req, res) {
  try {
    const { restaurantId } = req.params;

    const dish = await dishService.createDish(
      restaurantId,
      req.body
    );

    return res.status(201).json(dish);
  } catch (error) {
    return res.status(error.statusCode || 500).json({
      error: error.statusCode
        ? error.message
        : "Erro interno ao cadastrar prato"
    });
  }
}

async function list(req, res) {
  try {
    const { restaurantId } = req.params;

    const dishes = await dishService.listDishes(
      restaurantId
    );

    return res.json(dishes);
  } catch (error) {
    return res.status(500).json({
      error: "Erro ao consultar pratos"
    });
  }

}

module.exports = {
  create,
  list
};
```

O Controller não deve implementar diretamente as operações do Prisma.

Essa responsabilidade pertence ao Service.

Parte 7 — CRIE AS ROTAS

8. Implemente os endpoints

Abra:

```
src/modules/dishes/dish.routes.js
```

Implemente:

```
const express = require("express");
const router = express.Router();

const dishController = require("./dish.controller");

// Ajuste o caminho e o nome conforme
// o middleware criado nas aulas anteriores.
const authMiddleware = require("../auth/auth.middleware");

router.get(
  "/restaurants/:restaurantId/dishes",
  dishController.list
);

router.post(
  "/restaurants/:restaurantId/dishes",
  authMiddleware,
  dishController.create
);

module.exports = router;
```

Atenção: reutilize o middleware JWT que já existe no seu projeto. O caminho e a exportação acima são apenas um exemplo.

Depois, registre as rotas no **app.js**:

```
const dishRoutes = require("./modules/dishes/dish.routes");

app.use(dishRoutes);
```

Por que protegemos o POST?

O cadastro de pratos modifica informações da aplicação.

Por isso, exigiremos autenticação.

A consulta poderá continuar pública.

Nesta etapa, o JWT comprova que o usuário está autenticado. Verificar se ele possui autorização para alterar o cardápio de determinado restaurante seria uma evolução adicional.

Parte 8 — TESTE O CADASTRO

9. Cadastre um prato

Inicie a aplicação:

```
node server.js
```

Faça login e obtenha um JWT.

Depois execute:

```
POST http://localhost:3000/restaurants/1/dishes
```

Headers:

```
Content-Type: application/json
Authorization: Bearer SEU_TOKEN
```

Body:

```
{
  "name": "Pizza Margherita",
  "description": "Molho de tomate, queijo e manjericão",
  "price": 49.90
}
```

Resultado esperado:

```
201 Created
```

Importante: o restaurante de ID **1** precisa existir. Utilize um ID válido do seu banco.

10. Consulte o cardápio

Execute:

```
GET http://localhost:3000/restaurants/1/dishes
```

Resultado esperado:

```
[
  {
    "id": 1,
    "name": "Pizza Margherita",
    "description": "Molho de tomate, queijo e manjericão",
    "price": "49.90",
    "imageUrl": null,
    "restaurantId": 1
  }
]
```

O valor decimal pode ser serializado como string pelo Prisma.

Confirme que o prato está relacionado ao restaurante correto.

Parte 9 — PREPARE O UPLOAD DE IMAGENS

Agora que conseguimos cadastrar pratos, vamos implementar uma nova funcionalidade.

O restaurante poderá enviar uma imagem para representar cada prato do cardápio.

Entretanto, imagens muito grandes podem prejudicar o desempenho da aplicação.

Por isso, vamos processar a imagem antes de armazená-la.

11. Instale as bibliotecas

Utilizaremos duas bibliotecas:

Multer

- Recebe arquivos enviados por requisições HTTP multipart.

Sharp

- Permite redimensionar, converter e comprimir imagens.

Execute:

```
npm install multer sharp
```

12. Crie a estrutura de armazenamento

Dentro do projeto, crie:

```
uploads/
■■■ dishes/
```

A pasta será utilizada para armazenar as imagens otimizadas.

Nesta atividade, utilizaremos armazenamento local para simplificar a implementação.

Em um ambiente de produção com múltiplas instâncias ou sistema de arquivos temporário, precisaríamos avaliar um armazenamento externo, como S3.

Parte 10 — CRIE O SERVIÇO DE PROCESSAMENTO

13. Crie um serviço compartilhado

Dentro de **src/modules/shared/**, crie:

`image.service.js`

Por que utilizar **shared**?

Porque o processamento de imagens poderá ser reutilizado futuramente por outros módulos da EasyFood.

Por exemplo:

- Imagens dos pratos.
- Logotipos dos restaurantes.
- Imagens de perfil dos usuários.

Não queremos duplicar essa lógica em cada módulo.

14. Implemente o processamento

Abra:

`src/modules/shared/image.service.js`

Implemente:

```
const sharp = require("sharp");
const path = require("path");
const fs = require("fs/promises");
const crypto = require("crypto");

async function optimizeImage(buffer) {
  const filename = `${crypto.randomUUID()}.webp`;

  const directory = path.resolve(
    process.cwd(),
    "uploads",
    "dishes"
  );

  await fs.mkdir(directory, {
    recursive: true
  });

  const filePath = path.join(directory, filename);

  await sharp(buffer)
    .rotate()
    .resize({
      width: 800,
      withoutEnlargement: true
    })
    .webp({
      quality: 80
    })
    .toFile(filePath);

  return `/uploads/dishes/${filename}`;
}
```

```
module.exports = {

  optimizeImage
};
```

O que esse código faz?

1. Recebe a imagem original em memória.
2. Gera um nome único para o arquivo.
3. Corrige a orientação, quando aplicável.
4. Redimensiona a imagem para até 800 pixels de largura.
5. Converte para WebP com qualidade configurada.
6. Salva o arquivo otimizado.
7. Retorna o caminho da imagem.

O processamento acontece dentro do próprio servidor Node.js da EasyFood.

Não estamos utilizando Serverless nesta implementação.

Parte 11 — CONFIGURE O MULTER

15. Crie o middleware de upload

Dentro de **src/modules/shared/**, crie:

```
upload.middleware.js
```

Implemente:

```
const multer = require("multer");

const upload = multer({
  storage: multer.memoryStorage(),

  limits: {
    fileSize: 5 * 1024 * 1024
  },

  fileFilter: (req, file, callback) => {
    const allowedTypes = [
      "image/jpeg",
      "image/png",
      "image/webp"
    ];

    if (!allowedTypes.includes(file.mimetype)) {
      return callback(
        new Error("Formato de imagem não permitido")
      );
    }

    callback(null, true);
  }
});

module.exports = upload;
```

O que estamos configurando?

- O arquivo será recebido em memória.
- O limite será de 5 MB por arquivo.
- Aceitaremos JPEG, PNG e WebP.

O **file.mimetype** é uma informação fornecida pelo cliente e não garante, sozinho, que o arquivo seja uma imagem válida. O Sharp também deverá conseguir decodificá-lo. Em produção, seria necessário reforçar as

validações e os limites de segurança.

Parte 12 — ADICIONE A IMAGEM AO PRATO

16. Implemente a atualização da imagem

Abra:

`src/modules/dishes/dish.service.js`

Importe o serviço:

```
const imageService = require("../shared/image.service");
```

Adicione:

```
async function uploadDishImage(dishId, buffer) {
  const dish = await prisma.dish.findUnique({
    where: {
      id: Number(dishId)
    }
  });

  if (!dish) {
    const error = new Error("Prato não encontrado");
    error.statusCode = 404;
    throw error;
  }

  const imageUrl = await imageService.optimizeImage(
    buffer
  );

  return prisma.dish.update({
    where: {
      id: Number(dishId)
    },
    data: {
      imageUrl
    }
  });
}
```

Inclua a nova função no **module.exports**:

```
module.exports = {
  createDish,
  listDishes,
  uploadDishImage
};
```

O que acontece?

- Localizamos o prato.
- Enviamos a imagem ao serviço compartilhado.
- O serviço otimiza e salva o arquivo.
- Atualizamos o caminho da imagem no banco.

Parte 13 — CRIE O ENDPOINT DE UPLOAD

17. Adicione o Controller

No arquivo:

`src/modules/dishes/dish.controller.js`

Adicione:

```
async function uploadImage(req, res) {
  try {
    if (!req.file) {
      return res.status(400).json({
        error: "Nenhuma imagem enviada"
      });
    }

    const dish = await dishService.uploadDishImage(
      req.params.dishId,
      req.file.buffer
    );

    return res.json(dish);
  } catch (error) {
    return res.status(error.statusCode || 500).json({
      error: error.statusCode
        ? error.message
        : "Erro ao processar imagem"
    });
  }
}
```

Inclua **uploadImage** no **module.exports** do Controller.

18. Adicione a rota

No arquivo:

```
src/modules/dishes/dish.routes.js
```

Importe:

```
const upload = require("../shared/upload.middleware");
```

Adicione:

```
router.post(
  "/dishes/:dishId/image",
  authMiddleware,
  upload.single("image"),
  dishController.uploadImage
);
```

O middleware de autenticação deverá ser executado antes do upload.

19. Disponibilize as imagens

No **app.js**, adicione:

```
const path = require("path");

app.use(
  "/uploads",
  express.static(
    path.resolve(process.cwd(), "uploads")
  )
);
```

Assim, as imagens poderão ser acessadas pelo navegador.

Observação: nesta atividade, as imagens serão públicas. Em um sistema real, devemos avaliar controle de acesso, validação de arquivos e políticas de armazenamento.

Parte 14 — TESTE O UPLOAD

20. Envie uma imagem

Primeiro, cadastre um prato.

Depois execute:

```
POST http://localhost:3000/dishes/1/image
```

Headers:

```
Authorization: Bearer SEU_TOKEN
```

Body:

```
Tipo: form-data

Campo: image
Tipo: File
Valor: selecione uma imagem
```

Não configure manualmente o **Content-Type** com um boundary fixo. Deixe o Postman ou cliente HTTP gerar o cabeçalho multipart.

Resultado esperado:

```
200 OK
```

Exemplo de resposta:

```
{
  "id": 1,
  "name": "Pizza Margherita",
  "description": "Molho de tomate, queijo e manjericão",
  "price": "49.90",
  "imageUrl": "/uploads/dishes/arquivo.webp",
  "restaurantId": 1
}
```

O nome real do arquivo será gerado automaticamente.

21. Acesse a imagem otimizada

Abra no navegador o endereço retornado em **imageUrl**, utilizando o host da aplicação.

Exemplo:

```
http://localhost:3000/uploads/dishes/arquivo.webp
```

Confirme que a imagem é exibida.

Parte 15 — COMPARE AS IMAGENS

22. Verifique o resultado do processamento

Escolha uma imagem original e compare com a versão otimizada.

Registre:

```
Imagem original:
Formato:
Tamanho:
Dimensões:

Imagem otimizada:
Formato:
Tamanho:
Dimensões:
```

Responda

1. A imagem otimizada ficou menor?

Compare os tamanhos dos arquivos.

2. Qual transformação foi realizada?

Identifique o redimensionamento e a conversão de formato.

3. Por que isso pode melhorar a experiência do usuário?

Considere o tempo de transferência e o consumo de dados.

4. O arquivo sempre ficará menor?

Não necessariamente. O resultado depende da imagem original e das configurações de compressão.

Parte 16 — TESTE UMA FALHA

23. Simule um problema no processamento

Agora precisamos verificar como nossa aplicação se comporta quando algo dá errado.

Tente enviar:

- Um arquivo que não seja imagem.
- Uma imagem maior que o limite permitido.
- Uma requisição sem arquivo.

Resultado esperado

A aplicação não deverá encerrar inesperadamente.

Os erros devem ser tratados e retornar respostas HTTP adequadas.

Por exemplo:

```
400 Bad Request
```

Para um arquivo acima do limite, o status adequado é:

```
413 Payload Too Large
```

Para isso, trate os erros do Multer no middleware global de erros ou em um middleware específico de upload.

Reflita

Se o prato já foi cadastrado e a imagem falhar, devemos excluir o prato?

Não necessariamente.

O prato pode continuar cadastrado sem imagem, permitindo que o usuário tente enviar o arquivo novamente.

Esse comportamento recupera um conceito que já trabalhamos na atividade anterior:

Uma falha em uma operação secundária não precisa invalidar uma operação principal concluída.

Parte 17 — FLUXO FINAL

Nossa aplicação deverá seguir este fluxo:

Cadastro do prato

```
POST /restaurants/:restaurantId/dishes
            |
            v
    authMiddleware
            |
            v
    dish.controller.js
            |
            v
     dish.service.js
            |
            v
    prisma.dish.create()
            |
            v
        PostgreSQL
```

Upload da imagem

```
POST /dishes/:dishId/image
            |
            v
    authMiddleware
            |
            v
    upload.middleware.js
            |
            v
    dish.controller.js
            |
            v
     dish.service.js
            |
            v
    shared/image.service.js
            |
            v
       Sharp
            |
            v
    Imagem otimizada
            |
            v
    uploads/dishes/
            |
            v
    Prisma atualiza imageUrl
```

Estrutura final

```
src/
■■■ database/
■   ■■■ prisma.js
■■■ modules/
■   ■■■ restaurants/
■   ■   ■■■ restaurant.routes.js
■   ■   ■■■ restaurant.controller.js
■   ■   ■■■ restaurant.service.js
■   ■■■ auth/
■   ■   ■■■ ...
■   ■■■ dishes/
■   ■   ■■■ dish.routes.js
■   ■   ■■■ dish.controller.js
■   ■   ■■■ dish.service.js
■   ■■■ shared/
■       ■■■ email.service.js
■       ■■■ image.service.js
■       ■■■ upload.middleware.js
■■■ app.js
■■■ server.js

prisma/
■■■ schema.prisma

uploads/
■■■ dishes/
```

O **email.service.js** continua existindo e funcionando como na Atividade 6.

Parte 18 — REGISTRE A DECISÃO ARQUITETURAL

24. Crie um ADR

Durante a aula, discutimos diferentes alternativas para o processamento de imagens.

Agora registre a decisão adotada no projeto.

Crie um arquivo:

`docs/adr/ADR-002-processamento-de-imagens.md`

Se já existir um ADR-002, utilize a próxima numeração disponível.

O documento deverá conter:

Contexto

Por que a EasyFood precisa processar imagens?

Alternativas consideradas

- Processar dentro da API.
- Criar um serviço independente.
- Utilizar Serverless/FaaS.

Decisão

Manter o processamento dentro da API EasyFood.

Justificativa

Considere:

- Simplicidade.
- Baixo volume inicial de imagens.
- Infraestrutura já existente.

- Facilidade de implementação.
- Contexto acadêmico do projeto.

Consequências

Vantagens

- Menor complexidade inicial.
- Menos componentes para configurar.
- Aproveitamento da infraestrutura existente.

Limitações

- Consumo de CPU e memória da API.
- Possível impacto em outras requisições.
- Necessidade de reavaliar a solução caso o volume cresça.

Importante: nossa decisão não significa que Serverless seja inadequado. Significa que, neste momento, outra alternativa atende melhor às necessidades do projeto.

Atividade concluída quando

- [ ] A aplicação continua funcionando com as funcionalidades anteriores.
- [ ] O modelo **Dish** foi criado no Prisma.
- [ ] O relacionamento entre **Restaurant** e **Dish** funciona.
- [ ] A migração foi executada.
- [ ] O módulo **dishes** possui Route, Controller e Service.
- [ ] É possível cadastrar pratos.
- [ ] É possível consultar os pratos de um restaurante.
- [ ] O cadastro de pratos exige JWT.
- [ ] O Multer recebe as imagens.
- [ ] O Sharp realiza a otimização.
- [ ] A imagem otimizada é armazenada.
- [ ] O caminho da imagem é salvo no banco.
- [ ] É possível acessar a imagem pelo navegador.
- [ ] O tamanho original e o otimizado foram comparados.
- [ ] Falhas no upload não apagam o prato já cadastrado.
- [ ] O ADR foi criado justificando a decisão arquitetural.

EasyFood — Software Architecture & Design Patterns

Atividade 7 — Implementando o cardápio e o processamento de imagens